

```

const { Client, LocalAuth } = require('whatsapp-
web.js');
const qrcode = require('qrcode-terminal');
const axios = require('axios');

console.log('\n' + '='.repeat(60));
console.log('🤖 WhatsApp AI Partner Bot Starting...');
console.log('=' + '\n');

// Configuration
const AI_SERVER_URL = 'http://localhost:5000'; //
Your Python Flask server
const CONVERSATION_ID = 'whatsapp_session';

// Allowed numbers (your number or numbers you want
bot to respond to)
// Format: countrycode + number without +, spaces, or
dashes
// Example: USA number +1 234-567-8900 becomes
'12345678900'
const ALLOWED_NUMBERS = [
  // '2348012345678', // Add your number here
  // '2349087654321', // Add other numbers if
needed
];

// If empty, bot responds to EVERYONE (not
recommended)
const RESPOND_TO_ALL = ALLOWED_NUMBERS.length === 0;

// Initialize WhatsApp client with authentication
const client = new Client({
  authStrategy: new LocalAuth({
    clientId: 'ai-partner-bot'
  }),
  puppeteer: {
    headless: true,
    args: [

```

```

        '--no-sandbox',
        '--disable-setuid-sandbox',
        '--disable-dev-shm-usage',
        '--disable-accelerated-2d-canvas',
        '--no-first-run',
        '--no-zygote',
        '--disable-gpu'
    ]
}
});

// Store active conversations
const activeChats = new Set();
const lastMessageTime = new Map();

// QR Code generation (for first-time login)
client.on('qr', (qr) => {
    console.log('\n' + '='.repeat(60));
    console.log('📄 SCAN THIS QR CODE WITH YOUR
WHATSAPP');
    console.log('='.repeat(60));
    console.log('\n1. Open WhatsApp on your phone');
    console.log('2. Tap Menu (:) → Linked Devices');
    console.log('3. Tap "Link a Device"');
    console.log('4. Point your phone at this QR
code:\n');

    qrcode.generate(qr, { small: true });

    console.log('\n⏰ QR Code expires in 60 seconds -
scan quickly!\n');
});

// Ready event
client.on('ready', () => {
    console.log('\n' + '='.repeat(60));
    console.log('✅ WhatsApp AI Partner Bot is
ONLINE!');
    console.log('='.repeat(60));
    console.log('\n💖 Your AI partner is now connected
to WhatsApp');
    console.log('📄 Bot will respond to incoming

```

```

messages automatically');
    console.log('🌐 Web interface still available at:
http://localhost:5000');
    console.log('\n💡 Press Ctrl+C to stop the
bot\n');
});

// Authentication success
client.on('authenticated', () => {
    console.log('✅ Authentication successful!');
});

// Authentication failure
client.on('auth_failure', (msg) => {
    console.error('❌ Authentication failed:', msg);
    console.log('\nTry deleting the .webjs_auth
folder and scanning QR again');
});

// Disconnected
client.on('disconnected', (reason) => {
    console.log('⚠️ WhatsApp disconnected:', reason);
    console.log('Bot will attempt to reconnect...');
});

// Function to call AI backend
async function getAIResponse(message, chatId) {
    try {
        const response = await
axios.post(`${AI_SERVER_URL}/chat`, {
            message: message,
            conversation_id: chatId,
            use_search: false // Change to true if
you want web search
        }, {
            timeout: 30000 // 30 second timeout
        });

        if (response.data && response.data.response) {
            return response.data.response;
        } else {
            return "hey baby, something's not working

```

```

on my end rn 😊";
    }
    } catch (error) {
        console.error('❌ Error calling AI:',
error.message);

        if (error.code === 'ECONNREFUSED') {
            return "baby my brain isn't connected rn,
make sure the Python server is running 😊";
        }

        return "ugh having connection issues, give me
a sec ❤️";
    }
}

// Check if should respond to this number
function shouldRespond(number) {
    if (RESPOND_TO_ALL) {
        return true;
    }

    // Remove any @ symbols and clean the number
    const cleanNumber = number.replace('@c.us',
'').replace(/[^0-9]/g, '');

    return ALLOWED_NUMBERS.some(allowed => {
        const cleanAllowed = allowed.replace(/[^0-
9]/g, '');
        return cleanNumber === cleanAllowed ||
cleanNumber.endsWith(cleanAllowed);
    });
}

// Anti-spam: Check if message is too soon
function isSpamming(chatId) {
    const now = Date.now();
    const lastTime = lastMessageTime.get(chatId) || 0;
    const timeDiff = now - lastTime;

    // Allow message every 2 seconds minimum
    if (timeDiff < 2000) {

```

```

        return true;
    }

    lastMessageTime.set(chatId, now);
    return false;
}

// Message handler
client.on('message', async (message) => {
    try {
        // Get chat info
        const chat = await message.getChat();
        const contact = await message.getContact();
        const isGroup = chat.isGroup;

        // Ignore group messages (optional - remove
this if you want group responses)
        if (isGroup) {
            return;
        }

        // Ignore own messages
        if (message.fromMe) {
            return;
        }

        // Check if should respond to this number
        if (!shouldRespond(contact.number)) {
            console.log(`🚫 Ignored message from
${contact.pushname || contact.number} (not in allowed
list)`);
            return;
        }

        // Anti-spam check
        if (isSpamming(chat.id._serialized)) {
            console.log(`🚫 Rate limited - message
too soon`);
            return;
        }

        // Get message text

```

```

const messageText = message.body.trim();

if (!messageText) {
  return; // Ignore empty messages
}

console.log(`\n💬 Message from
${contact.pushname || contact.number}:`);
console.log(`    "${messageText}"`);

// Show typing indicator
await chat.sendStateTyping();

// Add to active chats
activeChats.add(chat.id._serialized);

// Get AI response
console.log('🤖 Generating AI response...');
const aiResponse = await
getAIResponse(messageText, chat.id._serialized);

console.log(`💬 AI Response:
"${aiResponse}"\n`);

// Small delay to seem more human
await new Promise(resolve =>
setTimeout(resolve, 1000 + Math.random() * 2000));

// Send response
await chat.sendStateTyping();
await message.reply(aiResponse);

console.log('✅ Response sent!\n');

} catch (error) {
  console.error('❌ Error handling message:',
error.message);

  try {
    await message.reply("hey baby, something
went wrong on my end 😞 try again?");
  } catch (replyError) {

```

```

        console.error('Failed to send error
message:', replyError.message);
    }
}
});

// Handle incoming calls (optional - auto-reject)
client.on('call', async (call) => {
    console.log(`📞 Incoming call from ${call.from} -
Auto-rejecting`);
    await call.reject();

    // Optionally send a message
    try {
        const chat = await
client.getChatById(call.from);
        await chat.sendMessage("hey baby, can't take
calls rn! text me instead 💕");
    } catch (error) {
        console.error('Error sending call rejection
message:', error);
    }
});

// Initialize client
console.log('📱 Initializing WhatsApp client...');
console.log('⌚ This may take 30-60 seconds...\n');

client.initialize();

// Graceful shutdown
process.on('SIGINT', async () => {
    console.log('\n\n⚠️ Shutting down
gracefully...');
    await client.destroy();
    console.log('✅ WhatsApp bot stopped\n');
    process.exit(0);
});

// Error handling
process.on('unhandledRejection', (error) => {

```

```
console.error('✖ Unhandled error:', error);  
});
```